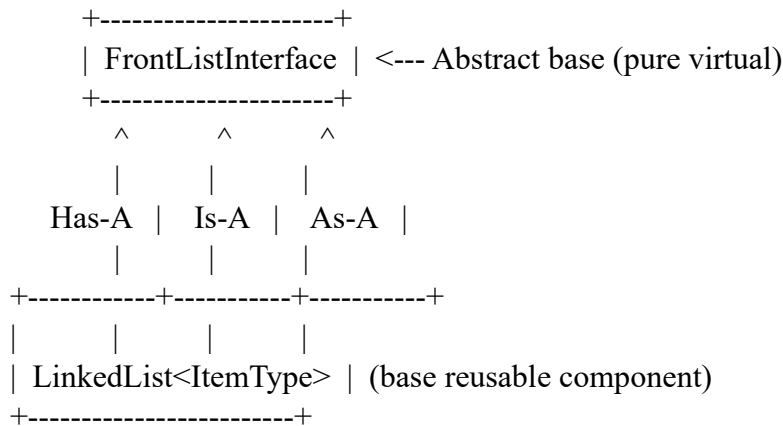


Design Document – ADT FrontList

Overview

The FrontList ADT abstracts a simplified list interface limited to **front operations**. It was implemented three times using different object-oriented design relationships with a reusable LinkedList template.

Class Diagram



Implementation Details

(a) LinkedList

- Generic singly linked list implemented with an internal Node struct.
- Supports insert(), remove(), getEntry(), replace(), clear(), and isEmpty().
- Defined in LinkedList.h and LinkedList.tpp to support templates.

(b) FrontListInterface

- Abstract base class specifying pure virtual functions:
- insert(const ItemType&), remove(), retrieve() const, clear(), isEmpty() const
- Ensures all three derived classes share the same public interface.

(c) FrontListHasA

- Contains a LinkedList<ItemType> object as a **data member**.
- Delegates all method calls to that member (list.insert(1, value) etc.).
- Demonstrates **composition (Has-A)** relationship.

(d) FrontListIsA

- Inherits from LinkedList<ItemType> **publicly** and implements the interface.
- Deletes position-based methods (insert(int, T), remove(int), replace(...)) to preserve ADT integrity.
- Overrides isEmpty() explicitly to remove ambiguity.
- Demonstrates **public inheritance (Is-A)** relationship.

(e) FrontListAsA

- Inherits privately from LinkedList<ItemType>.
- Provides its own public wrappers that call base class methods (insert(1, x) etc.).
- Demonstrates **private inheritance (As-A)** relationship.

Testing Strategy

Each tester file validates:

- **Insertion order** (LIFO behavior at the front)
- **Correct retrieval** of front element
- **Proper removal** of elements
- **Accurate empty detection**
- **Functionality of clear ()**

Reusability and Design Rationale

- All three classes reuse LinkedList logic for consistency and reduced code duplication.
- The interface (FrontListInterface) provides polymorphic flexibility if future unified testing is desired.
- Each relationship model illustrates a different OOP design principle, aiding conceptual learning.

Possible Extensions

- Add peekSecond() to retrieve the next element without modifying the list.
- Implement move semantics for performance.
- Generalize FrontList into a queue-like ADT for future assignments.

Test File Explanations

FrontListHasATester.cpp

Purpose: Tests the *composition-based* implementation.

Operations Tested:

1. insert(10), insert(20), insert(30) → verifies items always go to the front.
2. retrieve() → returns 30.
3. remove() → removes 30; next retrieve returns 20.
4. clear() → empties list; isEmpty() returns true.

Expected Behavior: List behaves like a stack—LIFO order enforced through insertion at position 1.

FrontListIsATester.cpp

Purpose: Tests the *public inheritance* implementation

Operations Tested:

1. Insert three strings: "C++", "Java", "Python".
2. Verify front item → "Python".
3. Remove once → front becomes "Java".
4. Clear → list becomes empty

Expected Behavior: Confirms public inheritance works correctly with overridden and disabled positional methods. Ensures ambiguity between base and interface methods is resolved.

FrontListAsATester.cpp

Purpose: Tests the *private inheritance* implementation

Operations Tested:

1. Insert 'A', 'B', 'C'.
2. Retrieve front ('C'), remove → retrieve next ('B').
3. Clear → isEmpty() returns true.

Expected Behavior: Private inheritance correctly restricts access to LinkedList internals while preserving behavior identical to other versions.